

Lab technician setup — Mastering MongoDB

Course: Mastering MongoDB (3 days) · Innovation In Software Corporation

Audience: The lab technician who prepares machines before class.

Click-by-click install: [WINDOWS-LAB-SETUP.md](#)

Prepare two separate environments. Participant laptops run Labs 1–5 and Lab 7. One shared replica set runs Lab 6 Steps 4–7. Do not turn a participant laptop into that replica set.

Environment	Who uses it	What it is
Participant laptop	Each learner, and the instructor's demo laptop	Windows standalone MongoDB on <code>localhost:27017</code>
Class replica set	Instructor and every participant, read-only	One three-member replica set, Atlas or self-hosted

Hand solution folders (`exercises/solution/`, `labN/solution/`) to the instructor only. Participant machines get the lab guides and `datasets\training_store\load.js`.

What to send the instructor when setup is finished

Send passwords in a private channel. Do not put a password, or a URI that contains a password, in email to the class, in a slide, or in this repository.

Participant laptops

- Count of machines ready, and any machine that failed a check below
- Confirmation that `mongod.cfg` has `bindIp: 127.0.0.1` and port `27017`
- Confirmation that `mongosh`, `mongodump`, and `mongoexport` print a version in a normal (non-Administrator) PowerShell window

Class replica set

- Host only: `mongodb+srv://<cluster-host>/training_store`
 - Class lab username (the same username for every participant)
 - Class lab password, privately
 - Instructor load-user username and password, privately, to the instructor only
 - The public IP you allowed
 - The test results in [Replica set sign-off](#), run as the class lab user from a participant laptop
-

1. Each participant laptop

Do this on every participant laptop and on the instructor's demo laptop. Same versions on every machine.

Software	Version this course records	Installer
Windows	10 or 11, 64-bit	—
MongoDB Community Server	8.3.11	Windows x64 MSI. Install as a Windows service named MongoDB.
MongoDB Compass	The copy bundled with the server MSI	Leave Install MongoDB Compass checked.
MongoDB Shell (mongosh)	2.12.0	Its own MSI. It is not inside the server MSI.
MongoDB Database Tools	100.19.0	Its own MSI. Required for Lab 7.
Course folder	Current class copy	Clone or the zip the instructor supplies. It must contain <code>datasets\training_store\load.js</code> .
PowerShell	Windows PowerShell	Labs are written for PowerShell, not Command Prompt or Git Bash.
VS Code or Cursor	Current	Recommended. The lab terminal is PowerShell in the editor.

Memory: 8 GB RAM. Disk: 10 GB free. The account that installs the service needs Administrator rights. After install, participants use a normal account.

Service settings

- Service name: MongoDB
- Run as Network Service. Do not set a service password.
- Listen on port 27017
- In `mongod.cfg`, `bindIp` stays `127.0.0.1`
- Leave access control off. The yellow startup warning in `mongosh` is expected. Do not enable authentication to clear it.
- Do not open port 27017 on the Windows firewall.
- Do not add `--repSet`. Do not create database users on this laptop.

Path

Database Tools install to `C:\Program Files\MongoDB\Tools\100\bin`. The MSI does not add that folder to Path.

1. Start → "environment variables" → **Edit environment variables for your account.**
2. Select **Path** → **Edit** → **New**.
3. Paste `C:\Program Files\MongoDB\Tools\100\bin` as the last entry.
4. Leave every other Path entry as it is.
5. Close any PowerShell window that was already open. Those windows keep the old Path.

Do not use `setx` to edit Path. It truncates long Path values.

In a **new** normal PowerShell window, if a command is still not recognized, reload Path once:

```
$env:Path = [Environment]::GetEnvironmentVariable("Path","Machine") + ";" +
[Environment]::GetEnvironmentVariable("Path","User")
```

Sign out of Windows and back in so later windows see Path without that line.

Laptop sign-off

Run these in a normal PowerShell window on each laptop. Every item must pass before Day 1.

```
Get-Service MongoDB
Test-NetConnection localhost -Port 27017
mongosh --version
mongodump --version
mongoexport --version
mongosh "mongodb://localhost:27017" --eval "db.runCommand({ ping: 1 })"
```

Check	Pass
Service MongoDB	Status Running
Test-NetConnection	RemoteAddress 127.0.0.1 and TcpTestSucceeded : True. A warning about ::1 (IPv6) can appear with that result. The IPv4 line is the one that matters.
mongosh --version	2.12.0
mongodump --version and mongoexport --version	100.19.0
Ping	{ ok: 1 }
Compass	Connects to <code>mongodb://localhost:27017</code>
mongod.cfg	bindIp: 127.0.0.1, port 27017
Course folder	datasets\training_store\load.js is on disk

`show dbs` on a new server lists `admin`, `config`, and `local`. `training_store` appears when Lab 2 loads it. You do not need to load it before Day 1.

2. The class replica set (Lab 6)

Labs 1–5 and Lab 7 never use this cluster. Lab 6 Steps 1–3 run on the participant laptop. Lab 6 Steps 4–7 read this replica set.

Build **one** replica set for the class:

- Three data-bearing members
- One PRIMARY and two SECONDARY members
- Each member has 1 vote, `health: 1`, and is not an arbiter
- An Atlas replica set is the expected shape. A self-hosted three-member replica set is acceptable.
- Use a cluster tier that lets a normal database user run `rs.status()`, `rs.conf()`, and `rs.printSecondaryReplicationInfo()`. Free and shared Atlas tiers often block those commands. A dedicated Atlas replica set is the reliable choice.
- Do not provision a sharded cluster for participants. Sharding and failover demos, if the instructor runs them, use a separate disposable cluster. Participants never receive that cluster's credentials, and nobody runs `rs.stepDown()` or `sh.shardCollection()` on the class replica set.

Two database users

Create database users. The Atlas (or ops) login is not a database user. Do not give participants an admin user.

User	Who receives the password	Privileges
Instructor load user	Instructor only	<code>readWrite</code> on database <code>training_store</code>
Class lab user	Instructor, who gives it to participants. One shared username.	<code>read</code> on <code>training_store</code> , and <code>clusterMonitor</code> so replica-set status commands succeed

The class lab user must not be able to insert, update, drop, or step down the primary.

Load the data before class

On a machine that has the course folder, sign in as the **instructor load user** and run this from the repository root. Type the password at the prompt. Leave the password out of the command.

```
mongosh "mongodb+srv://<cluster-host>/training_store" --username <instructor-load-user>
.\datasets\training_store\load.js
```

The script must print:

```
customers: 6
products: 13
orders: 17
reviews: 6
paid orders: 13
```

Participants do not run `load.js` against this shared cluster. They reload data only on their own laptop.

Network

- Add the lab's current public IP to the cluster access list. One lab egress address covers every laptop behind that address.
- Do not add `0.0.0.0/0`.
- Allow outbound DNS and outbound TCP **27017** from the lab network to the cluster.
- Participant laptops already have `mongosh`. Install nothing extra on those laptops for Lab 6.

Replica set sign-off

From a participant laptop, sign in as the **class lab user**:

```
mongosh "mongodb+srv://<cluster-host>/training_store" --username <lab-user>
```

Then, in `mongosh`:

```
db.hello()
rs.status()
rs.conf()
rs.printSecondaryReplicationInfo()
db.customers.countDocuments()
db.products.countDocuments()
db.orders.countDocuments()
db.reviews.countDocuments()
db.orders.countDocuments({ paymentStatus: "PAID" })
```

Check	Pass
<code>db.hello()</code>	Shows <code>setName</code> , a <code>hosts</code> list, and <code>primary</code>
<code>rs.status()</code>	Exactly one PRIMARY. Every reachable member has <code>health: 1</code> .
<code>rs.conf()</code>	Three voters, one vote each, no arbiter
<code>rs.printSecondaryReplicationInfo()</code>	A lag value for each secondary (0 seconds is a valid result)
Counts	6 customers, 13 products, 17 orders, 6 reviews, 13 paid orders

If `rs.status()` or `rs.conf()` returns “not authorized” for the class user, the tier or the role is wrong. Fix that before class. `db.hello()` alone is not enough for Steps 5–7.

What is ready on which day

When	Participant laptop	Class replica set
Before Day 1	Service, <code>mongosh</code> , Compass, course folder	—
Day 1 Lab 1 and Lab 2	Same laptop. Lab 2 loads <code>training_store</code> locally.	—
Day 2 Lab 3 and Lab 4	Same laptop	—
Day 3 Lab 5	Same laptop	—
Day 3 Lab 6	Steps 1–3 use the local <code>training_store</code>	Steps 4–7. Cluster is loaded and the class user can read status.
Day 3 Lab 7	<code>mongodump</code> and <code>mongorestore</code> on Path. Local server on <code>localhost:27017</code> .	Not used

Days 1 and 2 run offline after the installers are on the laptop. Lab 6 Steps 4–7 need the lab network path to the replica set.

Optional software

Install these only if the instructor asks:

Software	Used for
Git for Windows	Clone the course repository
Node.js 18+ or Python 3	Module 8 connection demo in <code>sample-app/</code>

The Module 8 demo uses the local server: `$env:MONGODB_URI = "mongodb://localhost:27017"`.

Leave these alone

- `bindIp` on participant laptops stays `127.0.0.1`.
- Access control on participant laptops stays off.
- Participant laptops stay standalone servers.
- The class replica set is read-only for participants.
- Passwords stay out of slides, worksheets, git, and class-wide email.